

AI/ML Task 2 - House Price Prediction Report

Anik Biswas | AI/ML Internship, Maincrafts Technology

1. Objective

In Task 1 I just loaded a dataset, trained one basic model and checked the accuracy - didn't really think too hard about the data itself. This one's more about the stuff that actually happens in real projects: scaling the data properly before training, trying more than one algorithm instead of assuming the first one is good enough, and comparing them with actual numbers instead of a gut feeling. I used the California Housing dataset that comes built into scikit-learn (`fetch_california_housing`), which has a bit over 20k rows and 8 features like median income, house age, rooms, population, and location - the target is the median house value for that area.

2. Dataset

20,433 rows (a few incomplete ones got dropped along the way), 8 input features - `MedInc`, `HouseAge`, `AveRooms`, `AveBedrms`, `Population`, `AveOccup`, `Latitude`, `Longitude` - and `HousePrice` (in \$100k units) as the target.

3. Methodology

First loaded the data and checked for nulls - turned out there weren't any, which was nice, one less thing to worry about. Before jumping straight into modeling I also ran a quick correlation check just to get a feel for the data, and median income (`MedInc`) stood out as the feature most correlated with price by far, which honestly makes sense - areas with higher income tend to have pricier houses.

After that I split the data into `X` (features) and `y` (target), then scaled `X` using `StandardScaler`. This step matters more than it sounds - some of these features are on completely different scales (`Population` is in the thousands, `AveRooms` is like single digits), so without scaling, the linear models especially would basically just end up weighting whatever feature has the biggest raw numbers. Did an 80/20 train-test split after that (`random_state=42`, mostly so I'd get the same results if I reran the notebook).

For the actual modeling part I trained three regressors on the same training set: Linear Regression as a baseline, Ridge Regression (`alpha=1.0`, adds L2 regularization) to see if regularizing helps at all, and a Decision Tree Regressor (`max_depth=5`, capped it so it doesn't just memorize the training data).

4. Results

Evaluated all three on the test set using RMSE and R^2 . RMSE tells you roughly how far off the predictions are on average (in the same \$100k units as the target), so lower is better. R^2 is more about how much of the variation in house prices the model is able to explain - closer to 1 is better.

Model	RMSE	MAE	R^2 Score
Decision Tree	0.7321	0.5318	0.6080
Linear Regression	0.7514	0.5400	0.5871
Ridge Regression	0.7514	0.5400	0.5872

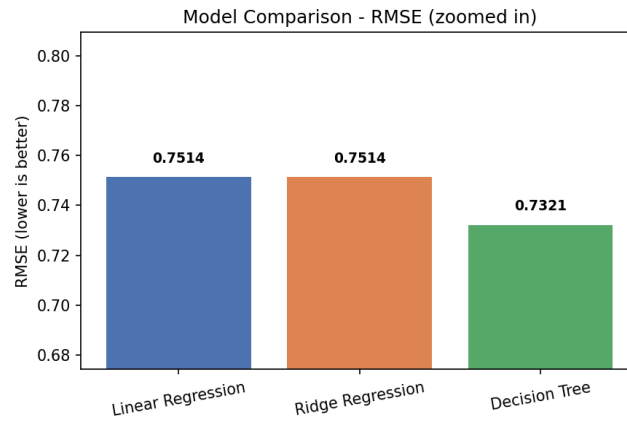


Fig 1. RMSE comparison across the three models (y-axis zoomed in since the values are close together).

The Decision Tree came out on top, with the lowest RMSE and best R^2 of the three. Linear and Ridge Regression basically tied with each other, which makes sense once you think about it - $\alpha=1.0$ isn't a very strong regularization strength, so Ridge doesn't really behave much differently from plain Linear Regression here. My guess for why the tree did a bit better is that it can actually pick up on non-linear patterns in the data (location clusters, income brackets, that kind of thing) that a straight-line model just can't represent, no matter how you scale the inputs.

That said, none of these models are amazing - R^2 around 0.6 means there's still a decent chunk of price variation none of them are explaining. I checked feature importance on the tree afterward and MedInc dominated everything else by a wide margin, with AveOccup and the lat/long coordinates coming in after that. Which tracks with the correlation heatmap I did earlier, so at least the results are consistent with each other. It's not too surprising though - this dataset has no info on school quality, crime, or the actual condition of the house, and those are things that move prices a lot in real life.

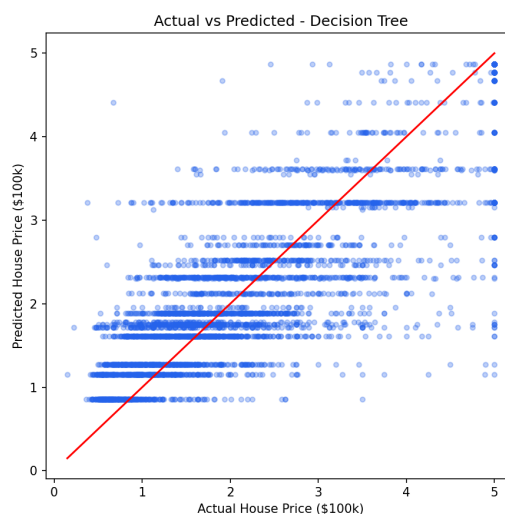


Fig 2. Actual vs predicted house prices on the test set for the Decision Tree (the best-performing model) - points closer to the red line mean better predictions.

5. Conclusion & Model Selection

Going with the **Decision Tree Regressor** as the final model for this task, since it beat the other two on both metrics. Saved it with `joblib` along with the scaler so I don't have to retrain everything from scratch if I want to reuse it later.

If I had more time I'd probably try tuning `max_depth` a bit more carefully instead of just picking 5, and maybe test something like Random Forest since it tends to beat a single decision tree in most cases - that's on the list for next time.